

Tutorial 13: The Sparring Ground

A practical local-server Screeps module

<docs/tutorials/13-sparring-ground.md>

Episode 13: The Sparring Ground

"A single wave is a smoke test, not a training loop," your collaborator says. "You've got a `curl` command that can throw another one at you any time you want. Let's actually use it that way – this is load testing, except the load is trying to kill your workers."

Episode 12 proved `role.defender.js` fights. It didn't prove it fights *well*. That takes losing a few times, in a row, on purpose – same as any real training loop, in code or anywhere else.

Goal

Turn the one-off invasion test from Episode 12 into a repeatable training loop, entirely on this repo's local server, with no internet connection required at any point:

- Understand why one test isn't iteration
- Run `scripts/sparring-loop.sh` to throw multiple waves at your colony back to back
- Watch the defender population strategy hold up, or not, under repeated pressure
- Understand the real cost of this kind of testing – it isn't a disposable sandbox, it's your actual colony

Prerequisites

Tutorial 12 is complete:

- `role.defender.js` exists and you've triggered at least one successful test invasion.
- `curl http://localhost:21025/local/api/sparring/health` returns `{"ok":1,...}` – the sparring-ground mod is loaded.

Step 1: Why One Test Isn't Enough

A single invader tells you the defender's code runs. It doesn't tell you whether two defenders are enough, whether the tower and the defenders coordinate well, or whether your energy economy can recover fast enough to rebuild before the next threat arrives. Those questions only show up under repeated pressure – which is exactly what a single manual click through the Invasion panel can't give you.

Step 2: Run the Sparring Loop

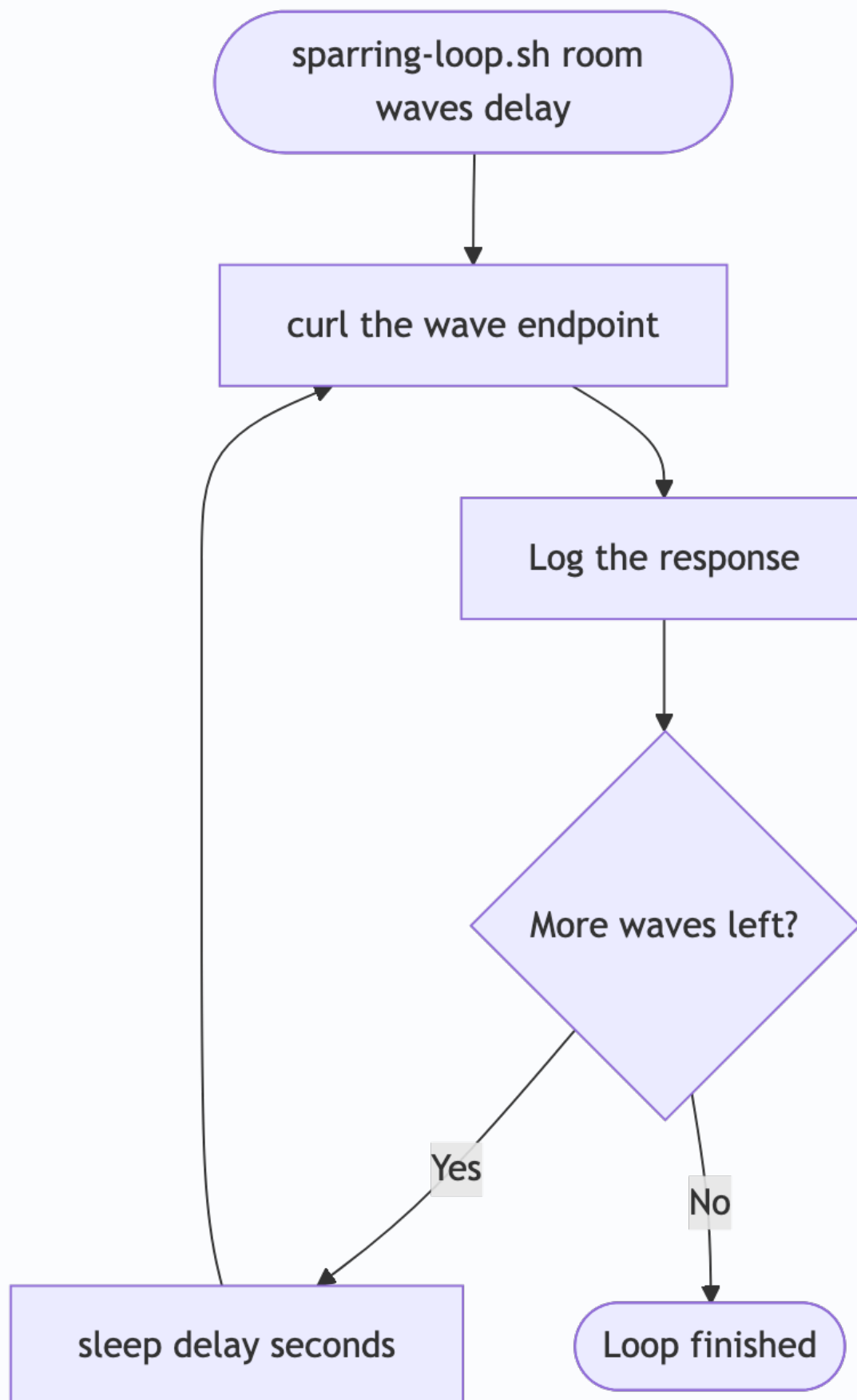
This repo ships a small script that calls the wave endpoint from Episode 7 on a timer:

```
./scripts/sparring-loop.sh <your-room-name> 3 30
```

The three arguments are room name, number of waves, and seconds between waves. That command throws three invasions at your room, thirty seconds apart, with no further input from you.

Checkpoint:

- Each wave logs a response in the terminal.
- Between waves, watch the client: does the defender population recover to full strength before the next wave lands, or is it still catching up?



 **Screenshot placeholder:** The terminal mid-loop, showing two or three wave responses already logged with a countdown to the next one – pairs well with a client screenshot of the colony taken at the same moment.

Step 3: Find the Breaking Point

Run the loop again with a shorter delay:

```
./scripts/sparring-loop.sh <your-room-name> 5 10
```

Somewhere between "plenty of recovery time" and "no recovery time," this colony's defense either holds or it doesn't. Finding that line is the actual point of this episode – not proving the code works once, but learning where it stops working.

Checkpoint: you can state, in one sentence, what specifically broke first – spawn queue falling behind, tower running out of energy, defenders dying before reinforcements arrive, or something else.

Step 4: Debug the Failure Like Episode 10 Taught

Whatever broke, treat it like a real bug: write a structured prompt using the five-part shape from Episode 10 (expected, actual, relevant code, what you tried, what "fixed" looks like), work through it with your collaborator, and log the session in `docs/journal/`.

This is the loop this whole episode exists to create: run waves, observe a failure, debug it properly, fix it, run waves again. Repeat as many times as you want – the tool doesn't get tired.

Step 5: Understand What This Tool Actually Costs You

This is not a disposable sandbox. Every wave you run costs your real colony real energy, real creep lifetimes, and real structure hit points. A defender that dies in a test is a defender you have to re-spawn with real resources. A tower drained defending against a test wave is a tower that's briefly weaker against a real one.

That's a genuine tradeoff worth sitting with: this local setup gives you unlimited, free, offline *attempts* – but each attempt is against your one real colony, not a disposable copy of it. Compare that to what a separate, match-based environment would offer (a fresh map every time, nothing persistent to protect) and you can see why some studios build both kinds of tools instead of just one. For this season, the always-available, zero-dependency version is the one worth having.

Troubleshooting

If a wave doesn't land, re-check Episode 7's troubleshooting – the room needs a neutral, unclaimed exit for the engine to spawn into.

If `scripts/sparring-loop.sh` fails with a permissions error, run `chmod +x scripts/sparring-loop.sh` once.

If the colony's economy never recovers between runs, that's not a bug in the script – pause the loop, let the colony rebuild for a few minutes of real ticks, and resume. The loop is a tool for finding limits, not a demand to test past them every single time.

Completion Criteria

You are done when:

- You've run `scripts/sparring-loop.sh` with at least two different delay settings.
- You can name the specific failure point you found at the shorter delay.
- You've logged at least one debugging session from this episode in `docs/journal/`.
- You can explain, in your own words, why this tool costs real colony resources per test and what tradeoff that represents.

Learning Notes

After completing the tutorial, write down:

- What was the shortest delay between waves your colony's defense could still handle?
- Was the bottleneck combat (not enough defenders) or economy (not enough energy to rebuild them)? How would you find out for certain?

- If you ran this loop overnight unattended, what's the worst thing that could happen to the colony?
- What would a truly disposable version of this tool need that the current one doesn't have?

Next: Episode 14 – Open the Gates

You've played the game from every angle this season offers: economy, infrastructure, logistics, defense, expansion, and now repeated, deliberate combat testing.

"There's one role left," your collaborator says, "and it's not a creep. It's you. Somebody has to run the league the next architect drops into – congratulations, you've been promoted to management."

See `docs/roadmap.md` for the full season.